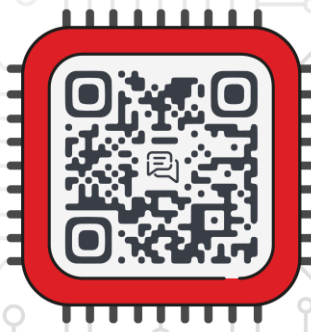




8



11



1320 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

Java Collection

عملي مشترك



RB Informatics; 18/12/2024

برمجة ٣

مراجعات.. ➤

Collection

← هو object يمثل container يحوي مجموعة من الـ objects الأخرى. (نتعامل مع كل شيء على أنه object)

Collection Interfaces

← **تغلف** أنواع مختلفة من الـ Collections وتمثل أنواع من الـ data، التي هي من الـ Framework.

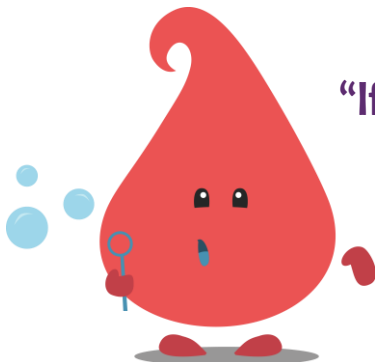
← حيث الـ Interfaces التي سنتعامل معها هي:

Collection
Map

يجب الانتباه إلى أن جميعها Interfaces أي كل

method ضمنها بحاجة إلى @Override عند

استخدامها



“If you cannot do great things, Do small things in a great way.”

-Napoleon Hill

أهم ال methods الموجودة في Collection Interfaces

<< Basic Operations >>

- `int size();`
- `boolean isEmpty();`
- `boolean contains(object element);`
- `boolean add(E element);`
- `boolean remove(object element);`
- `Iterator <E> iterator;`

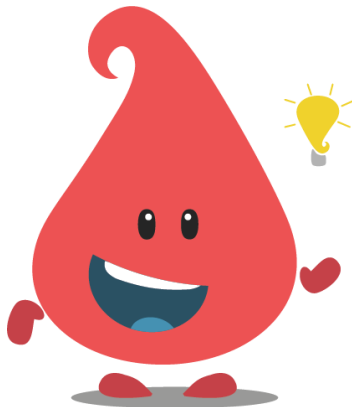
أي يعطينا object من نوع iterator

<< Bulk Operations >>

- `boolean containsAll(Collection <?> c);`
- `boolean addAll(Collection <? extends E> c);`
- `boolean removeAll(Collection <?> c);`
- `void clear();`
- `boolean retainAll(Collection <?> c);`

<< Array Operations >>

- `toArray();`



نتنبيه إلى أن جميع التوابع السابقة هي

abstract

Iterable Interfaces & Iterators

← ترثه كل الـ Interfaces وكل الـ classes وذلك لأن الـ `collection interface` ترث منه، مهمته هي تأمين `iterator` للـ `collections` لذا يحوي abstract method واحدة هي `iterator`.

Iterator

← نستخدمه كلما أردنا تعداد عناصر وذلك من أجل `collection` ما.
 ← عند استخدامه يجب أن نقوم بـ `import java.util.*` وكذلك عند استخدام `collection framework`.
 ← يوجد فيه 3 توابع أساسية هي:

```
public boolean hasNext();
```

```
public object next();
```

```
public object remove();
```

ArrayList

← نعرف الـ `ArrayList` بالشكل التالي:

```
ArrayList<__> list = new ArrayList<> ();
```

نكتب نوع الـ `object`
 قد يكون `interger`,
`character`, etc..

object

يمكن تركه فارغ

← لإضافة عنصر إلى الـ `ArrayList` نكتب:

```
list.add(__);
```

نمرر العنصر المراد
 إضافته

وتكون الإضافة في نهاية المصفوفة

↩ بعض التوابع ضمن ArrayList:

- `list.get(index);` --> يعيد العنصر في المكان `index`
- `list.getFirst();` --> يعيد العنصر الأول
- `list.getLast();` --> يعيد العنصر الأخير
- `list.remove(index);` --> يقوم بحذف العنصر الذي في المكان `index`
- `list.remove(Object);` --> يقوم بحذف أوبجكت معين من المصفوفة
- `list.clear();` --> يقوم بحذف جميع عناصر المصفوفة

ملاحظة!

لإضافة مصفوفة ما مثلا:

```
Integer [ ] arr = new Integer [ ] { 1, 2, 3, 4};
```

إلى ال ArrayList. نكتب:

```
Collections.addAll( List , arr );
```

المصفوفة التي نريد
الإضافة عليها

المصفوفة المراد إضافتها

Believe you CAN and you're
half way there..



HashSet

← نعرف الـ HashSet بالشكل التالي:

```
HashSet <__> set = new HashSet < > ();
```

نوع الـ object

← لإضافة عنصر إلى الـ HashSet نكتب:

```
set.add(__);
```

العنصر المراد إضافته



تقوم الـ HashSet بحذف العناصر المتشابهة تلقائياً.
أي لا تحوي الـ HashSet على عناصر متكررة.

HashMap

← نعرف الـ HashMap بالشكل التالي:

```
HashMap <__, __> map = new HashMap < > ();
```

يأخذ قيمتين

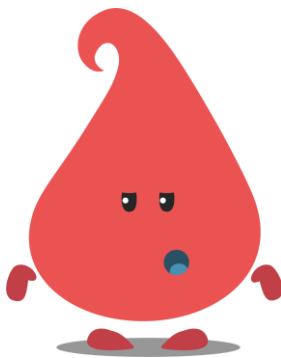
key & value

ولكل منهما type خاص به

ويمكن ترك الأقواس فارغة لجعلها تستقبل أي type

← لإضافة عنصر إلى الـ HashMap نكتب مثلاً:

```
map.put( 2, "Monday");  
map.put( 4, "Wednesday");
```



ويصبح الـ HashMap بالشكل التالي:

```
map = {  
    2 : "Monday",  
    4 : "Wednesday"  
}  
// Key : Value
```

مثال مهم

لدينا النص التالي:

`String text = "abcdabcbdaabcbdf";`

نريد معرفة عدد مرات تكرار كل حرف character.

```

1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         String text = "abcdabcbdaabcbdf";
6         HashMap<Character, Integer> map = new HashMap<>();
7
8         for (Character c : text.toCharArray()) {
9             map.put(c, map.getOrDefault(c, 0) + 1);
10        }
11
12        System.out.println(map);
13    }
14 }

```

NOT
<char , int>

OUTPUT

{ a=4, b=3, c=3, d=3, f=1 }

السطر (8): حلقة تكرارية تمر على جميع محارف النص text وتخزنهم في c.السطر (9): نقوم بوضع الثنائي:

c : map.getOrDefault(c,0)+1

في الـ HashMap

حيث c يمثل الحرف من النص text، و map.getOrDefault(c,0) يعيد لنا

قيمة العنصر ذو الـ key=c في حال وجوده في الـ HashMap أو يعيد لنا قيمة

افتراضية Default وهي -0-

ملاحظة!

يوجد توابع لمعرفة أكثر عنصر متكرر وأقل عنصر متكرر وتستخدم كالتالي:

Collections.max(map.values());

Collections.min(map.values());

تمارين ➤

1. اكتب الصف الرئيسي ArrayToCollection يحوي التابع الرئيسي Main والذي يقوم بقراءة مصفوفة من دخل المستخدم ثم يقوم بتحويلها إلى Collection من نمط قائمة List ثم طباعة عناصر هذه القائمة وطباعة العنصر الأكبر Max والأصغر Min

```
1 import java.util.*;
2 import java.io.*;
3 public class ArrayToCollection{
4     public static void main(String args[]) throws IOException {
5         BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
6         System.out.println("How many elements you want to add to the array: ");
7         int n = Integer.parseInt(in.readLine());
8         String[] name = new String[n];
9         for(int i = 0; i < n; i++) {
10             name[i] = in.readLine();
11         }
12         List<String> list = Arrays.asList(name);
13         System.out.println();
14         for(String li: list) {
15             System.out.print(li + " ");
16         }
17         System.out.println("max: " + Collections.max(list));
18         System.out.println("min: " + Collections.min(list));
19     }
20 }
```

السطر (1,2) تضمين المكتاب الضرورية

السطر (3,4) تعريف الصف ArrayToCollection مع التابع main الرئيسي

السطر (5,11) تعريف مصفوفة وقراءة عناصرها من دخل المستخدم

السطر (12,16) تحويل المصفوفة إلى قائمة وطباعة عناصرها

السطر (17,18) طباعة العنصر الأكبر والأصغر ضمن القائمة

2. اكتب الصف الرئيسي HashSetCollection يحوي التابع الرئيسي Main الذي يقوم بتعريف Collection من نمط HashSet وتهيئته بأسماء الألوان ثم طباعة عناصره وبعدها قم بحذف أحد هذه العناصر وطباعة العناصر من جديد بعد الحذف وأيضاً حجم الـ collection بعد الحذف

```

1 import java.util.*;
2 public class HashSetCollection {
3     public static void main(String [] args) {
4         System.out.println("Collection Example!\n");
5         int size;
6         HashSet <String>collection = new HashSet <String>();
7         String str1 = "Yellow", str2 = "White", str3 = "Green",
str4 = "Blue";
8         Iterator iterator;
9         collection.add(str1);
10        collection.add(str2);
11        collection.add(str3);
12        collection.add(str4);
13        System.out.print("Collection data: ");
14        iterator = collection.iterator();
15        while (iterator.hasNext()){
16            System.out.print(iterator.next() + " ");
17        }
18        System.out.println();
19        collection.remove(str2);
20        System.out.println("After removing [" + str2 +
21        System.out.print("Now collection data: ");
22        iterator = collection.iterator();
23        while (iterator.hasNext()){
24            System.out.print(iterator.next() + " ");
25        }
26        System.out.println();
27        size = collection.size();
28        if (collection.isEmpty()){
29            System.out.println("Collection is empty");
30        } else {
31            System.out.println("Collection size: " + size);
32        }
33        System.out.println();
34    }
35 }

```



السطر (1) تضمين مكتبة الـ Util

السطر (2,3) تعريف الصف

HashSetCollection مع التابع main

الرئيسي

السطر (4,12) تعريف المتحولات الضرورية

وتعبئة الـ HashSet بالقيم المطلوبة

السطر (13,17) طباعة عناصر الـ HashSet

السطر (18,25) حذف عنصر من

الـ HashSet وطباعة عناصرها بعد الحذف

السطر (26,33) طباعة حجم

الـ HashSet بعد التأكد أنها ليست فارغة

3. اكتب الصف الرئيسي SubListInList يحوي التابع الرئيسي main الذي يقوم بتعريف قائمة من مجموعة من الأسماء ولتكن أرقام مثلاً ثم يقوم بتعريف قائمة جزئية وإيجاد مكان وجود هذه القائمة الجزئية ضمن القائمة الأساسية، ثم استبدال أحد عناصر القائمة الرئيسية أينما وجد بعنصر آخر.

```
1 import java.util.*;
2 public class SubListInList {
3     public static void main(String[] args) {
4         List list = Arrays.asList("one Two three Four five six one three Four".split(" "));
5         System.out.println("List :"+list);
6         List sublist = Arrays.asList("three Four".split(" "));
7         System.out.println("SubList :"+sublist);
8         System.out.println("indexOfSubList: " +
9             Collections.indexOfSubList(list, sublist));
10        System.out.println("lastIndexOfSubList: " +
11            Collections.lastIndexOfSubList(list, sublist));
12        Collections.replaceAll(list, "one", "hundread");
13        System.out.println("replaceAll: " + list);
14    }
15 }
```

السطر (1) تضمين مكتبة ال Util

السطر (2,3) تعريف الصف SubListInList مع التابع main الرئيسي

السطر (4,7) تعريف القائمة الرئيسية والجزئية المطلوبين

السطر (9,8) طباعة مكان أول ظهور للقائمة الجزئية في القائمة الرئيسية

السطر (10,11) طباعة آخر ظهور للقائمة الجزئية في القائمة الرئيسية

السطر (12,13) استبدال كلمة one بكلمة hundread في كل الأماكن

ضمن القائمة الرئيسية



4. اكتب الصف الرئيسي ReverseList يحوي التابع الرئيسي Main ويقوم بتعريف قائمة من String ثم يقوم بعكس هذه القائمة من العنصر الأخير إلى العنصر الأول وطباعة القائمة بعد العكس

```
1 import java.util.ArrayList;
2 import java.util.Collections;
3 import java.util.List;
4 import java.util.ListIterator;
5 public class ReverseList {
6     public static void main(String[] args) {
7         String[] coins = { "A", "B", "C", "D", "E" };
8         List l = new ArrayList();
9         for (int i = 0; i < coins.length; i++)
10             l.add(coins[i]);
11         ListIterator liter = l.listIterator();
12         System.out.println("Before reversal");
13         while (liter.hasNext())
14             System.out.println(liter.next());
15         Collections.reverse(l);
16         liter = l.listIterator();
17         System.out.println("After reversal");
18         while (liter.hasNext())
19             System.out.println(liter.next());
20     }
21 }
```

السطر (1,4) تضمين المكتبات الضرورية

السطر (5,6) تعريف الصف ReverseList مع

التابع main الرئيسي

السطر (7,10) تعريف القائمة وتعبئة عناصرها

السطر (11,14) طباعة عناصر القائمة

السطر (15,20) عكس القائمة وطباعة

عناصرها بعد العكس

وتشاء أنت من الأمانى نجمة،
ويشاء ربك أن يناولك القمر..



5. اكتب الصف الرئيسي HashMapTest يحوي التابع الرئيسي Main الذي يقوم بتعريف HashMap تحوي الأرقام من 1 حتى 7 كمفاتيح والأيام من السبت للجمعة كقيم ثم طباعة العناصر بالشكل المناسب ثم قم بطباعة اليوم رقم 2، ثم قم بحذف اليوم رقم 3 وأطبع العناصر بعد الحذف

```

1 import java.util.HashMap;
2 import java.util.Map;
3 import java.util.Iterator;
4 import java.util.Set;
5 public class HashMapTest {
6     public static void main(String args[]) {
7         HashMap<Integer, String> hmap = new HashMap<>();
8         hmap.put(1, "Saturday");
9         hmap.put(2, "Sunday");
10        hmap.put(3, "Monday");
11        hmap.put(4, "Tuesday");
12        hmap.put(5, "Wednesday");
13        hmap.put(6, "Thursday");
14        hmap.put(7, "Friday");
15        Set set = hmap.entrySet();
16        Iterator iterator = set.iterator();
17        while(iterator.hasNext()) {
18            Map.Entry mentry = (Map.Entry)iterator.next();
19            System.out.print("key is: " + mentry.getKey() + " & Value is: ");
20            System.out.println(mentry.getValue());
21        }
22        String var= hmap.get(2);
23        System.out.println("Value at index 2 is: "+var);
24        hmap.remove(3);
25        System.out.println("Map key and values after removal:");
26        Set set2 = hmap.entrySet();
27        Iterator iterator2 = set2.iterator();
28        while(iterator2.hasNext()) {
29            Map.Entry mentry2 = (Map.Entry)iterator2.next();
30            System.out.print("Key is: "+mentry2.getKey() + " & Value is: ");
31            System.out.println(mentry2.getValue());
32        }
33    }
34 }

```

السطر (1,4) تضمين المكتبات

الضرورية

السطر (5,6) تعريف الصف

الرئيسي HashMapTest والتابع

الرئيسي main

السطر (7,14) تعريف ال

hashmap وتعبئتها بالعناصر

المطلوبة

السطر (15,21) طباعة عناصرها

على شكل مفتاح وقيمة

السطر (22,23) طباعة اليوم ذو

الرقم 2

السطر (24,32) حذف اليوم ذو

الرقم 3 وطباعة العناصر من

جديد بعد الحذف